

EXAMEN PRÁCTICO — LARAVEL

Plataforma de Gestión de Música

Módulo	Desarrollo Web en Entorno Servidor
Duración	3:20 horas
Puntuación total	10 puntos
Material permitido	Documentación oficial de Laravel (laravel.com/docs), apuntes propios
Entrega	Plataforma AEDUCAR

Puesta en marcha del proyecto

Sigue estos pasos en orden antes de escribir una sola línea de código. Una instalación mal hecha puede costarte tiempo muy valioso.

Paso 1 — Crear el proyecto Laravel

```
laravel new musica-app
```

Cuando el instalador pregunte:

- **Starter kit** → selecciona `Breeze`
- **Breeze stack** → selecciona `Blade with Alpine`
- **Testing framework** → selecciona `PHPUnit`
- **Base de datos** → selecciona `MySQL`

Si no tienes el instalador de Laravel disponible, usa Composer:

```
composer create-project laravel/laravel musica-app
```

En ese caso instala Breeze manualmente en el Paso 3.

Paso 2 — Configurar el archivo `.env`

Abre `.env` en la raíz del proyecto y edita el bloque de base de datos:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=musica_app
DB_USERNAME=root
DB_PASSWORD=
```

Ajusta `DB_USERNAME` y `DB_PASSWORD` según tu entorno. Con XAMPP o Laragon por defecto el usuario es `root` y la contraseña está vacía.

Crea la base de datos en MySQL antes de continuar:

```
CREATE DATABASE musica_app CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Paso 3 — Instalar Laravel Breeze (solo si no lo hiciste en el Paso 1)

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install
npm run build
```

Paso 4 — Ejecutar migraciones y enlazar Storage

```
php artisan migrate
php artisan storage:link
```

`storage:link` crea el enlace simbólico necesario para que las imágenes subidas sean accesibles públicamente. Sin este paso las portadas no se mostrarán.

Paso 5 — Inicializar el repositorio Git

```
git init
git add .
git commit -m "Instalación inicial"
```

Crea un repositorio **público** en GitHub y conéctalo:

```
git remote add origin https://github.com/TU_USUARIO/musica-app.git
git push -u origin main
```

Haz push con frecuencia durante el examen. Solo se evaluará el código que esté en GitHub al finalizar el tiempo.

Paso 6 — Arrancar el servidor

```
php artisan serve
```

Abre `http://localhost:8000` y comprueba que la página de inicio de Breeze carga correctamente antes de continuar.

PARTE 1 — Tutorial de despliegue: CRUD + Autenticación y Roles (4 puntos)

En el **sobre de código** encontrarás todos los fragmentos necesarios para construir la aplicación paso a paso. Cada paso indica el comando Artisan para generar el fichero y el fragmento del sobre que debes colocar dentro.

Sigue los pasos en el orden indicado. Cada uno construye sobre el anterior.

Paso 1 — Migración de la tabla `albums`

```
php artisan make:migration create_albums_table
```

Abre el fichero generado en `database/migrations/` y coloca el **Fragmento A** dentro del método `up()` y el método `down()`.

```
php artisan migrate
```

Paso 2 — Modelo `Album`

```
php artisan make:model Album
```

Sustituye el contenido de `app/Models/Album.php` con el **Fragmento B**.

Paso 3 — Seeder de datos de prueba

```
php artisan make:seeder AlbumSeeder
```

Sustituye el contenido de `database/seeder/AlbumSeeder.php` con el **Fragmento C**.

Añade esta línea dentro del método `run()` de `database/seeder/DatabaseSeeder.php`:

```
$this->call(AlbumSeeder::class);
```

```
php artisan db:seed
```

Paso 4 — Campo `role` en la tabla `users`

```
php artisan make:migration add_role_to_users_table
```

Abre el fichero generado y coloca el **Fragmento D** dentro de los métodos `up()` y `down()`.

```
php artisan migrate
```

Paso 5 — Métodos de rol en el modelo `User`

Abre `app/Models/User.php` y añade el **Fragmento E** dentro de la clase, junto al resto de métodos. No elimines nada del fichero original.

Paso 6 — Seeder de usuario administrador

```
php artisan make:seeder AdminSeeder
```

Sustituye el contenido de `database/seeder/AdminSeeder.php` con el **Fragmento F**.

Añade también en `DatabaseSeeder.php` :

```
$this->call(AdminSeeder::class);
```

```
php artisan db:seed --class=AdminSeeder
```

Comprueba que puedes hacer login en `http://localhost:8000/login` con `admin@musica.com` / `password` .

Paso 7 — Middleware de administrador

Crea el fichero `app/Http/Middleware/IsAdmin.php` y coloca dentro el **Fragmento G**.

Registra el alias en `bootstrap/app.php` , dentro del método `withMiddleware()` :

```
->withMiddleware(function (Middleware $middleware) {  
    $middleware->alias([  
        'admin' => \App\Http\Middleware\IsAdmin::class,  
    ]);  
})
```

Paso 8 — Form Request de validación

```
php artisan make:request StoreAlbumRequest
```

Sustituye el contenido de `app/Http/Requests/StoreAlbumRequest.php` con el **Fragmento H**.

Paso 9 — Controlador de álbumes

```
php artisan make:controller AlbumController
```

Sustituye el contenido de `app/Http/Controllers/AlbumController.php` con el **Fragmento I**.

Paso 10 — Vistas Blade

Crea la carpeta `resources/views/albums/` y coloca cada fragmento en su fichero:

Fragmento	Fichero
Fragmento J	<code>resources/views/albums/index.blade.php</code>
Fragmento K	<code>resources/views/albums/show.blade.php</code>
Fragmento L	<code>resources/views/albums/create.blade.php</code>
Fragmento M	<code>resources/views/albums/edit.blade.php</code>

Paso 11 — Rutas

Abre `routes/web.php` y añade el **Fragmento N** al final del fichero, respetando los grupos de rutas de Breeze que ya existen.

Paso 12 — Verificación final

Antes de pasar a la Parte 2, comprueba que todo funciona:

- `http://localhost:8000/albums` muestra el listado con los datos del seeder
- Un usuario registrado puede crear un álbum nuevo
- Solo el creador o el admin pueden editar o eliminar un álbum
- Los formularios muestran errores en español si se envían vacíos
- Las imágenes de portada se muestran correctamente

Si algo falla, revisa los mensajes de error antes de continuar.

Criterios de valoración — Parte 1

Criterio	Puntos
Migración albums ejecutada correctamente	0,25
Migración role ejecutada correctamente	0,25
Modelo Album con <code>\$fillable</code> y relaciones correctos	0,25
Seeders de álbumes y administrador funcionales	0,25
Métodos <code>isAdmin()</code> y <code>albums()</code> añadidos al modelo User	0,25
Middleware <code>IsAdmin</code> creado y registrado en <code>bootstrap/app.php</code>	0,50
Form Request con reglas y mensajes en español	0,50
Controlador: métodos <code>index</code> , <code>show</code> , <code>create</code>	0,25
Controlador: métodos <code>store</code> y <code>update</code> con gestión de imagen	0,50
Controlador: método <code>destroy</code> y <code>authorizeAccess</code>	0,25

Vistas index y show funcionales	0,25
Vistas create y edit con errores de validación visibles	0,25
Rutas correctamente definidas y protegidas	0,25

PARTE 2 — Resolución de errores (3 puntos)

En el sobre encontrarás un controlador con **3 errores deliberados**.

Para cada error debes:

1. Indicar en qué método se encuentra
2. Explicar qué está mal y por qué
3. Escribir el código corregido

Entrega las respuestas en un fichero `errores_resueltos.md` en la raíz del proyecto.

Pistas (una por error, en desorden)

- Encadenar dos métodos en ese orden no es posible en Eloquent.
- La condición de autorización tiene un operador lógico incorrecto: permite actuar a quien no debería.
- Una variable calculada nunca llega a la vista.

Criterios de valoración — Parte 2

Criterio	Puntos
Cada error identificado, explicado y corregido correctamente	1 × 3 = 3 puntos

Se otorgará puntuación parcial (0,5) si se identifica y explica correctamente el error pero la corrección es incompleta.

PARTE 3 — Nueva funcionalidad: Buscador con filtros avanzados (3 puntos)

Implementa un **buscador de álbumes con filtros avanzados** completamente desde cero, sin código de apoyo.

Descripción funcional

El buscador debe permitir buscar álbumes aplicando uno o varios de los siguientes filtros de forma simultánea:

Filtro	Descripción
Texto libre	Busca coincidencias en el título del álbum y en el nombre del artista
Género	Desplegable con los géneros existentes en la base de datos
Año desde / hasta	Rango de años de lanzamiento
Valoración mínima	Álbumes con <code>average_rating</code> igual o superior al valor elegido (1–5)

Ordenación	Por título (A-Z), año (más reciente primero) o valoración (mayor primero)
-------------------	---

Requisitos técnicos

- **Ruta:** GET /albums/search con nombre `albums.search`, definida **antes** del resource de álbumes en `web.php`
- **Método:** `search(Request $request)` dentro de `AlbumController`
- **Eloquent:** usa `when()` para aplicar cada filtro de forma condicional
- **Paginación:** `->paginate(10)->withQueryString()` para mantener los filtros al cambiar de página
- **Vista:** `resources/views/albums/search.blade.php`
 - Formulario con todos los filtros usando método GET
 - Los campos conservan los valores introducidos al mostrar resultados (usa `request('campo')`)
 - Muestra el número total de resultados encontrados
 - Si no hay resultados, muestra un mensaje informativo
 - Incluye un enlace al buscador visible desde `albums.index`

Esqueleto orientativo

```
public function search(Request $request): View
{
    $query = Album::query();

    // Aplica aquí los filtros con when()

    $albums = $query->paginate(10)->withQueryString();
    $genres = Album::distinct()->orderBy('genre')->pluck('genre');

    return view('albums.search', compact('albums', 'genres'));
}
```

Criterios de valoración — Parte 3

Criterio	Puntos
Ruta correcta, bien nombrada y en el orden adecuado	0,25
Método <code>search()</code> con <code>when()</code> aplicado correctamente	0,50
Filtros de texto, género, rango de años y valoración funcionales	1,00
Ordenación funcional	0,25
Vista con formulario, resultados y contador	0,50
Paginación persistente y valores del formulario conservados	0,50

Entrega

```
git add .  
git commit -m "Examen Laravel - [Tu nombre]"  
git push
```

Envía el enlace al repositorio a AEDUCAR y el código en un .zip por el canal indicado por el profesor antes de que finalice el tiempo.

El código que no esté en AEDUCAR al cierre del examen no será evaluado.
